DATA 260 — Homework 6

Xu Wang · MongoDB Document API and Agent Memory

GitHub repository: https://github.com/xuwang-goldenwater/data260-9275

Collaborators with access: Sbnikitha, supriyaselvanganesan.

Section 0 — Configuration

Value	Definition	Mine
SID4	last four digits of student ID	9275
PORT_BASE	8000 + (SID4 mod 900)	8275
PREFIX	"s" + SID4	s9275
SEED	SID4	9275
VERIFY_SEED	260000 + SID4	269275
DOMAIN_ID	SID4 mod 8	3 — Grocery supply and recall notices

Item	Value
Hardware	MacBook Pro, Apple M1, 8 GB unified memory
Local model	qwen2.5:3b via Ollama (temperature 0, seed 9275, num_ctx 8192), called through src/model_client.py; 100% GPU, ~11.6 tokens/s
Embeddings	sentence-transformers/all-MiniLM-L6-v2 on CPU
Database	MongoDB 7.0 in Docker (container s9275-mongo), database s9275_doc
Tagged commit	d34f933, 34 passed; later commits add only docs and the report.

Part 1 — MongoDB Document API

Same recall-notice domain as HW4/HW5, now stored as documents. The firm is embedded in the notice instead of a firm_id foreign key, ids are ObjectIds, and validation happens twice: Pydantic in the API and a `$jsonSchema` validator in the database. Code: `code/doc_app/` — `config.py` (server configuration), `db.py` (PyMongo connection), `schemas.py` (Pydantic models), `routers/notices.py` (route handlers), `main.py` (app, middleware).

I. Database

Collection `s9275_doc.recall_notices`: required `product_name` (1–100 chars), optional `reason`, enum `status` (ongoing / completed / terminated, default ongoing), enum `category` (4 fixed values), required `recall_date` (BSON date), unique `recall_number` (F-NNNN-YYYY), embedded `firm`, automatic `created_at` / `updated_at`.

code/doc_app/init_db.py — database-side validation

```
NOTICE_VALIDATOR = {
    "$jsonSchema": {
        "bsonType": "object",
        "required": ["product_name", "recall_number", "status", "category",
                     "recall_date", "created_at", "updated_at"],
```

```
        "additionalProperties": False,          # unknown fields are rejected
        "properties": {
            "_id": {"bsonType": "objectId"},
            "product_name": {"bsonType": "string", "minLength": 1,
                             "maxLength": config.PRODUCT_NAME_MAX},
            "reason": {"bsonType": ["string", "null"], "maxLength": config.REASON_MAX},
            "recall_number": {"bsonType": "string",
                              "pattern": config.RECALL_NUMBER_PATTERN},
            "status": {"enum": list(config.STATUSES)},
            "category": {"enum": list(config.CATEGORIES)},
            "recall_date": {"bsonType": "date"},
            ...
```

code/doc_app/init_db.py — create or update the validator (re-runnable), indexes

```
    if name in db.list_collection_names():
        db.command("collMod", name, validator=NOTICE_VALIDATOR,
                   validationLevel="strict", validationAction="error")
        print(f"[ok] {config.DB_NAME}.{name} exists -> validator replaced (collMod)")
    else:
        db.create_collection(name, validator=NOTICE_VALIDATOR,
                             validationLevel="strict", validationAction="error")
        print(f"[ok] created {config.DB_NAME}.{name} with $jsonSchema validator")
    col = db[name]

    # 2) indexes: unique recall_number (-> 409 in the API), newest-first listing
    col.create_index([("recall_number", ASCENDING)], unique=True, name="uniq_recall_number")
    col.create_index([("recall_date", DESCENDING)], name="ix_recall_date")
    print("[ok] indexes:", sorted(col.index_information()))
```

II. API implementation

code/doc_app/routers/notices.py — CRUD handlers

```
@router.post("", response_model=NoticeOut, status_code=status.HTTP_201_CREATED)
def create_notice(payload: NoticeIn):
    doc = payload.model_dump()                  # validated dict, status default filled in
    doc["created_at"] = doc["updated_at"] = now_utc()
    col = get_notices()
    with mongo_errors("creating the notice"):
        result = col.insert_one(doc)
        return to_out(col.find_one({"_id": result.inserted_id}))


@router.get("/{notice_id}", response_model=NoticeOut)
def get_notice(notice_id: str):
    oid = parse_id(notice_id)
    with mongo_errors("fetching the notice"):
        doc = get_notices().find_one({"_id": oid})
    if doc is None:
        raise HTTPException(status_code=404, detail=f"Notice {notice_id} not found.")
    return to_out(doc)


@router.put("/{notice_id}", response_model=NoticeOut)
def update_notice(notice_id: str, payload: NoticeIn):
    oid = parse_id(notice_id)
    changes = payload.model_dump()
    changes["updated_at"] = now_utc()
    with mongo_errors("updating the notice"):
        doc = get_notices().find_one_and_update(
            {"_id": oid}, {"$set": changes}, return_document=ReturnDocument.AFTER)
    if doc is None:
        raise HTTPException(status_code=404, detail=f"Notice {notice_id} not found.")
    return to_out(doc)


@router.delete("/{notice_id}", status_code=status.HTTP_204_NO_CONTENT)
def delete_notice(notice_id: str):
    oid = parse_id(notice_id)
```

```
    with mongo_errors("deleting the notice"):
        result = get_notices().delete_one({"_id": oid})
    if result.deleted count == 0:
        raise HTTPException(status code=404, detail=f"Notice {notice id} not found.")
    return Response(status_code=status.HTTP_204_NO_CONTENT)
```

code/doc_app/routers/notices.py — PyMongo errors to HTTP status codes

```
@contextmanager
def mongo_errors(action: str):
    try:
        yield
    except DuplicateKeyError as exc:
        key = (exc.details or {}).get("keyValue", {})
        raise HTTPException(status_code=409,
                            detail=f"Duplicate value {key}: recall_number must be unique.")
    except WriteError as exc:
        if exc.code == VALIDATION FAILED:     # should not happen: Pydantic checks first
            raise HTTPException(status code=422,
                                detail="Document rejected by the database $jsonSchema
validator.")
        raise HTTPException(status_code=500, detail=f"Database error while {action}.")
    except (ServerSelectionTimeoutError, ConnectionFailure):
        raise HTTPException(status_code=503, detail="Database unavailable (MongoDB not
reachable).")
    except PyMongoError:
        raise HTTPException(status_code=500, detail=f"Database error while {action}.")
```

code/doc_app/main.py — JSON-body middleware and CORS

```
@app.middleware("http")
async def require_json_body(request: Request, call_next):
    if request.method in JSON METHODS:
        content type = request.headers.get("content-type", "")
        if not content_type.startswith("application/json"):
            return JSONResponse(status_code=415, content={
                "error": "unsupported_media_type",
                "detail": "Send the request body as JSON (Content-Type: application/json)."})
    return await call_next(request)

app.add_middleware(CORSMiddleware, allow_origins=["*"], allow_credentials=False,
                   allow_methods=["*"], allow_headers=["*"])
```

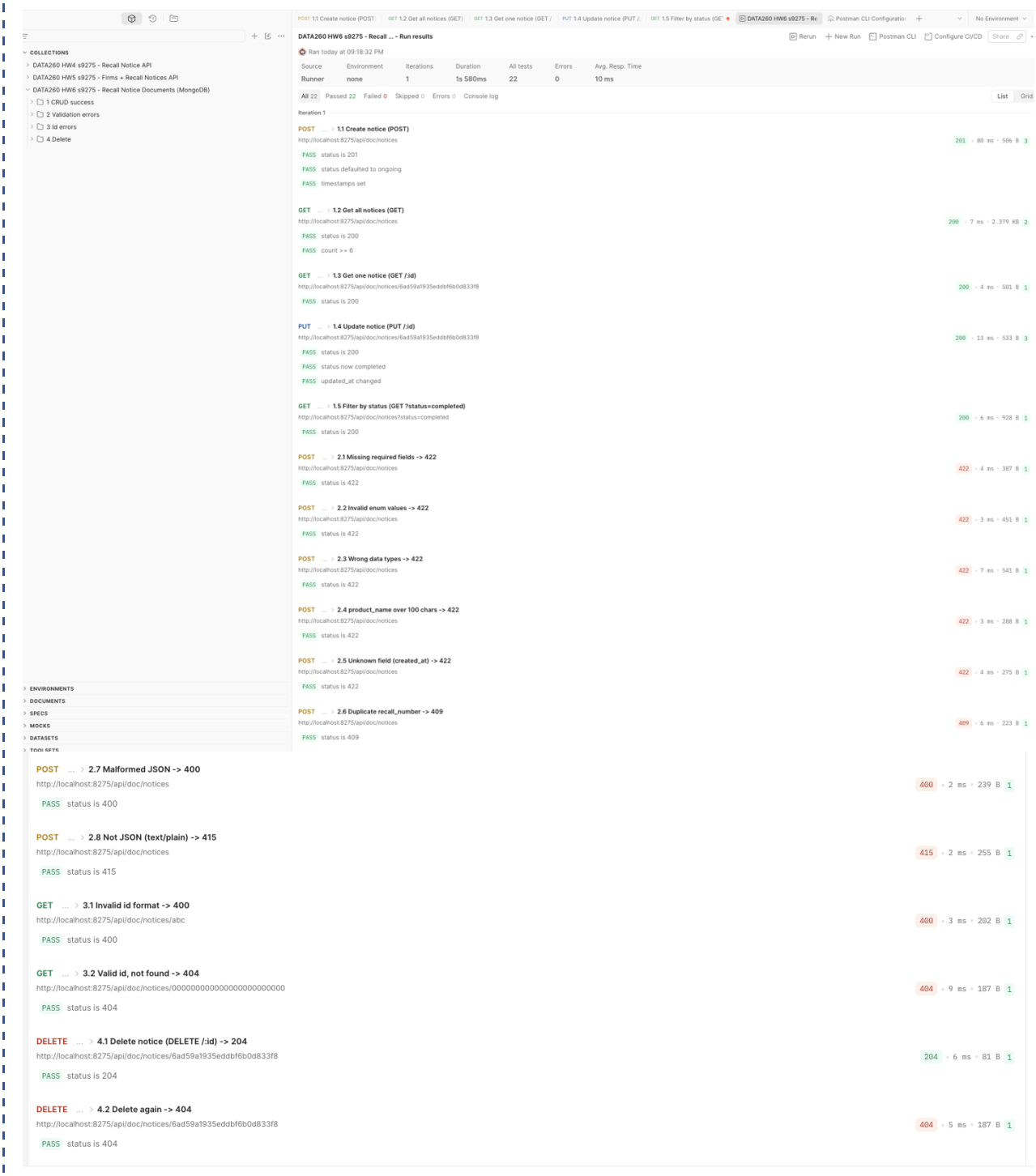


Figure 1.1. Postman Collection Runner — all 17 requests: CRUD (1.x, 4.x) and validation/error cases (2.x, 3.x) with status codes and passing tests

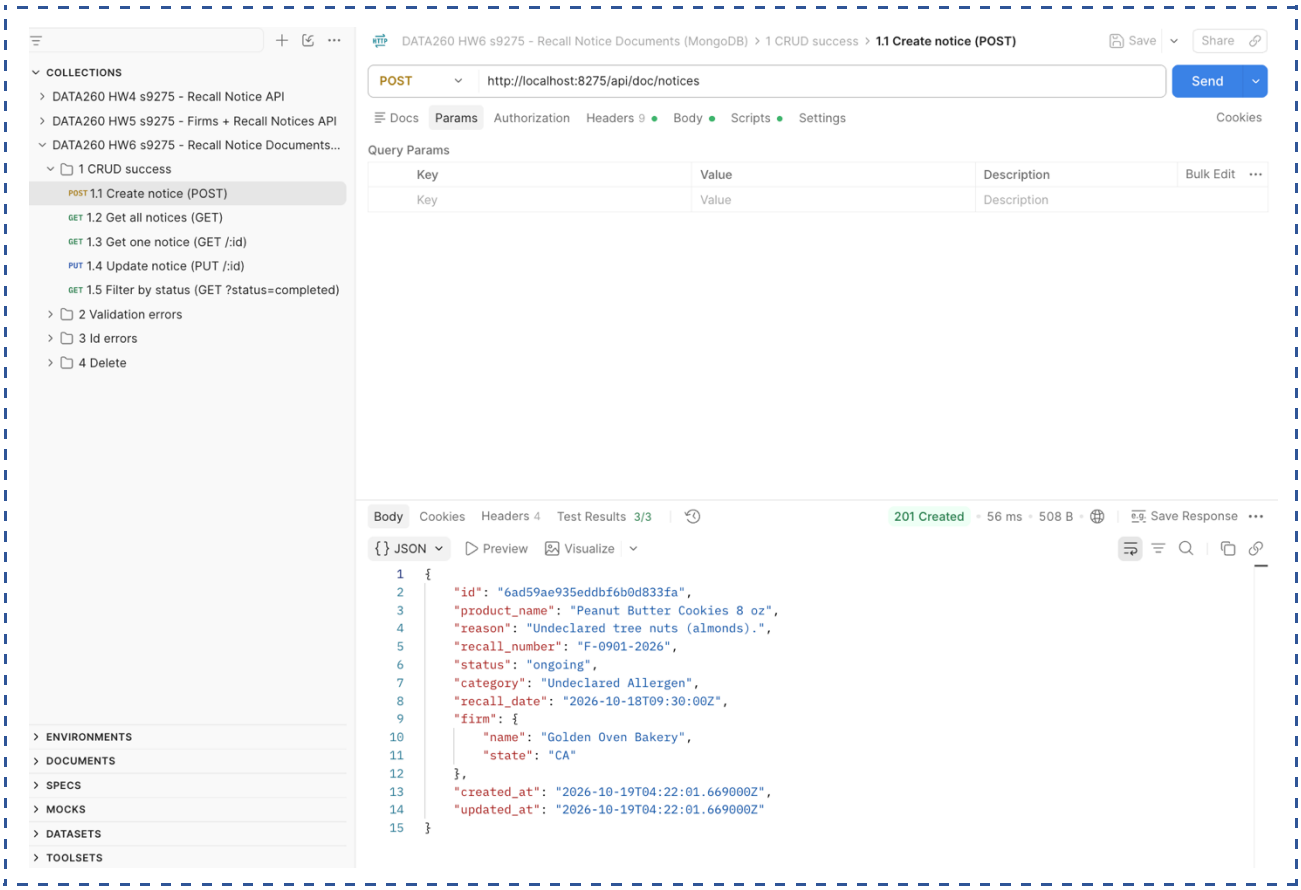


Figure 1.2. Postman 1.1 POST create → 201 (status defaults to ongoing, created_at / updated_at set)

III. Data validation

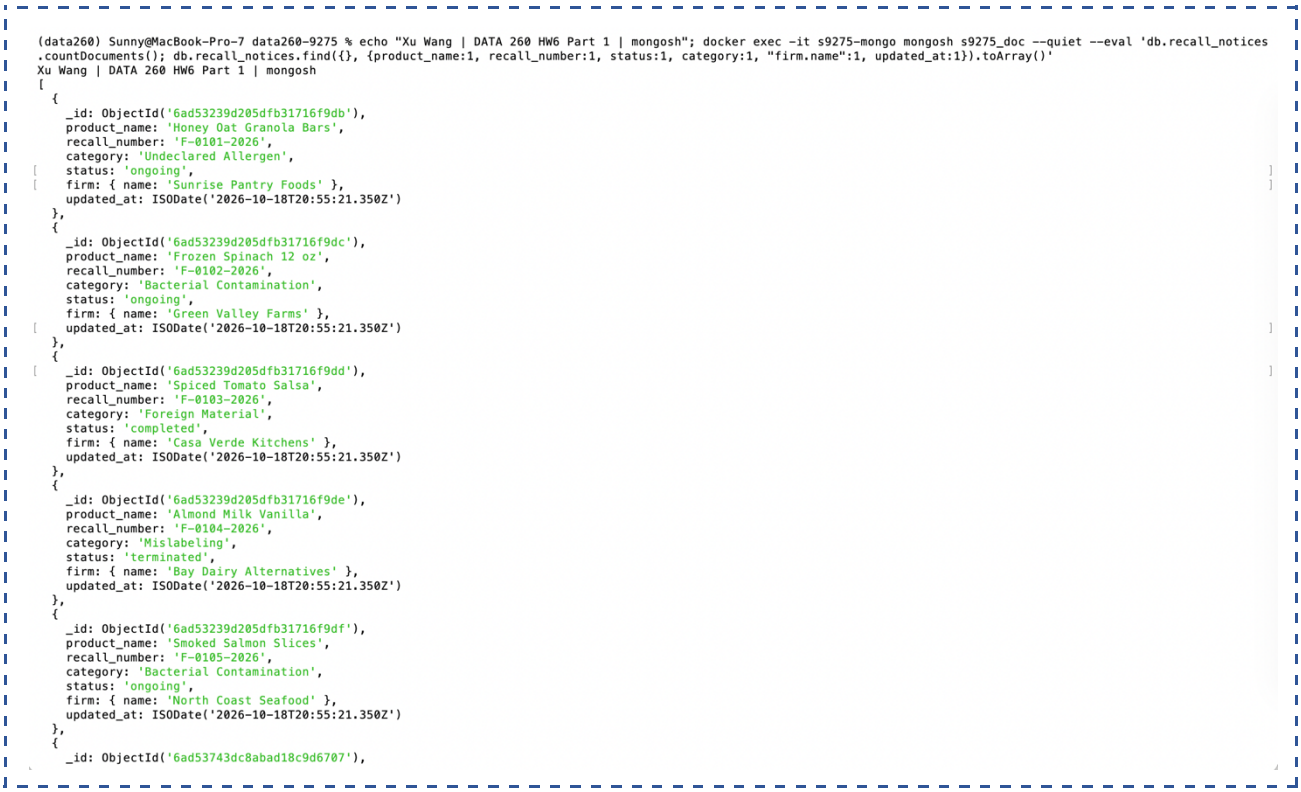
code/doc_app/schemas.py — request schema

```
class NoticeIn(BaseModel):
    model_config = ConfigDict(extra="forbid", str_strip_whitespace=True)


    product_name: str = Field(min_length=1, max_length=config.PRODUCT_NAME_MAX)
    reason: Optional[str] = Field(default=None, max_length=config.REASON_MAX)
    recall_number: str = Field(pattern=config.RECALL_NUMBER_PATTERN)
    status: Status = config.DEFAULT_STATUS
    category: Category
    recall date: datetime
    firm: Optional[FirmIn] = None


    @field validator("recall date")
    @classmethod
    def assume_utc(cls, value: datetime) -> datetime:
        """A time sent without a zone is taken as UTC, so storage is consistent."""
        return value.replace(tzinfo=timezone.utc) if value.tzinfo is None else value
```

Validation tests are requests 2.1–2.8 and 3.1–3.2 in the Collection Runner result above: missing required fields, invalid enum values and wrong data types → 422 with field-level messages; duplicate recall_number → 409; malformed JSON → 400; non-JSON body → 415; invalid id → 400; unknown id → 404.



```
(data260) Sunny@MacBook-Pro-7 data260-9275 % echo "Xu Wang | DATA 260 HW6 Part 1 | mongosh"; docker exec -it s9275-mongo mongosh s9275_doc --quiet --eval 'db.recall_notices
.countDocuments(); db.recall_notices.find({}, {product_name:1, recall_number:1, status:1, category:1, "firm.name":1, updated_at:1}).toArray()'
Xu Wang | DATA 260 HW6 Part 1 | mongosh
[
  {
    _id: ObjectId('6ad53239d205dfb31716f9db'),
    product_name: 'Honey Oat Granola Bars',
    recall_number: 'F-0101-2026',
    category: 'Undeclared Allergen',
    status: 'ongoing',
    firm: { name: 'Sunrise Pantry Foods' },
    updated_at: ISODate('2026-10-18T20:55:21.350Z')
  },
  {
    _id: ObjectId('6ad53239d205dfb31716f9dc'),
    product_name: 'Frozen Spinach 12 oz',
    recall_number: 'F-0102-2026',
    category: 'Bacterial Contamination',
    status: 'ongoing',
    firm: { name: 'Green Valley Farms' },
    updated_at: ISODate('2026-10-18T20:55:21.350Z')
  },
  {
    _id: ObjectId('6ad53239d205dfb31716f9dd'),
    product_name: 'Spiced Tomato Salsa',
    recall_number: 'F-0103-2026',
    category: 'Foreign Material',
    status: 'completed',
    firm: { name: 'Casa Verde Kitchens' },
    updated_at: ISODate('2026-10-18T20:55:21.350Z')
  },
  {
    _id: ObjectId('6ad53239d205dfb31716f9de'),
    product_name: 'Almond Milk Vanilla',
    recall_number: 'F-0104-2026',
    category: 'Mislabeling',
    status: 'terminated',
    firm: { name: 'Bay Dairy Alternatives' },
    updated_at: ISODate('2026-10-18T20:55:21.350Z')
  },
  {
    _id: ObjectId('6ad53239d205dfb31716f9df'),
    product_name: 'Smoked Salmon Slices',
    recall_number: 'F-0105-2026',
    category: 'Bacterial Contamination',
    status: 'ongoing',
    firm: { name: 'North Coast Seafood' },
    updated_at: ISODate('2026-10-18T20:55:21.350Z')
  },
  {
    _id: ObjectId('6ad53743dc8abad18c9d6707'),
```

Figure 1.3. mongo shell: recall_notices collection with its sample documents (the notice created through the API is shown in Figure 1.2)

Part 2 — Short-, Long-, and Episodic Memory

Same database and connection, three new collections: `messages` {user_id, session_id, role, content, created_at}, `summaries` {user_id, session_id or null, scope 'session'|'user', text, created_at}, `episodes` {user_id, session_id, fact, importance 0..1, embedding (384 floats), created_at}. Settings: SHORT_TERM_N = 12, SUMMARIZE_EVERY_USER_MSGS = 4, lifetime summary every 2 session summaries, top-k = 3.

Endpoints

code/doc_app/routers/chat.py — POST /api/chat, GET /api/memory/{user_id}, GET /api/aggregate/{user_id}

```
@router.post("/api/chat")
def chat(payload: ChatIn):
    with mongo_errors("running the chat turn"):
        try:
            result = memory.chat_turn(payload.user_id, payload.message,
                                      payload.session_id, payload.short_term_n,
                                      payload.window_policy)
        except Exception as exc:
            if model_unavailable(exc):
                raise HTTPException(status_code=503,
                                    detail="Model unavailable: is Ollama running (ollama
serve)?")
            raise
    # Part 3-1: every /api/chat call is logged (tokens per section, latency)
    try:
        memory.log_turn(config.CHAT_LOG_PATH, result, {"source": "api"})
    except Exception as exc:                  # logging must never break the chat
        print(f"[warn] chat log not written: {exc}")
    result.pop("_sections", None)
    return result


@router.get("/api/memory/{user_id}")
def get_memory(user_id: str = UserId):
    with mongo_errors("reading memory"):
        return memory.memory_view(user_id)


@router.get("/api/aggregate/{user_id}")
def get_aggregate(user_id: str = UserId):
    with mongo_errors("aggregating messages"):
        return memory.aggregate_view(user_id)
```

Memory logic

code/doc_app/memory.py — one chat turn (retrieve before extract, so a message cannot retrieve its own new facts)

```
    user_doc = save_message(user_id, session_id, "user", message)                 # 1
    window = load_short_term(user_id, session_id, n, exclude_id=user_doc["_id"])  # 2
    window = apply_window_policy(window, window_policy)
    session summary = latest summary(user id, "session", session id)             # 3
    lifetime summary = latest summary(user id, "user")
    facts = retrieve_episodes(user_id, message, config.TOP_K)                     # 4
    messages, sections = build_prompt(session_summary, lifetime_summary,          # 5
                                      window, message, facts)
    resp = get_model().complete(messages, max_tokens=config.REPLY_MAX_TOKENS)    # 6
    reply = resp.text.strip()
    save_message(user_id, session_id, "assistant", reply)                         # 7
    new_facts, extract_usage = extract_facts(message)                             # 8
    stored = store_episodes(user_id, session_id, new_facts)
    summary_events = maybe_summarize(user_id, session_id)                         # 9
```

memory.py — episode extraction guards (questions removed, facts must be grounded in the message)

```
def statement_part(message: str) -> str:
    import re
    sentences = re.split(r"(?<=[.!?])\s+", message.strip())
    return " ".join(s for s in sentences if s and not s.rstrip().endswith("?"))


def grounded(fact: str, source: str) -> bool:
    fact words =  words(fact)
    if not fact_words:
        return False
    stems = {w[:4] for w in _words(source)}
    hits = sum(1 for w in fact_words if w[:4] in stems)
    return hits / len(fact_words) >= 0.5
```

memory.py — summarization trigger (session summary every M user messages, lifetime summary)

```
def maybe_summarize(user_id: str, session_id: str) -> List[Dict[str, Any]]:
    events: List[Dict[str, Any]] = []
    if count_user_messages(user_id, session_id) % config.SUMMARIZE_EVERY_USER_MSGS != 0:
        return events
    events.append(summarize_session(user_id, session_id))
    n session = summaries col().count documents({"user id": user id, "scope": "session"})
    if n session % config.LIFETIME EVERY SESSION SUMMARIES == 0:
        events.append(condense_lifetime(user_id))
    return events
```

memory.py — embedding and top-k retrieval by cosine similarity

```
def embed(texts: List[str]) -> np.ndarray:
    vectors = get_embedder().encode(texts, normalize_embeddings=True, convert_to_numpy=True)
    return np.asarray(vectors, dtype=np.float32)


def retrieve_episodes(user_id: str, query_text: str, k: int) -> List[Dict[str, Any]]:
    docs = list(episodes_col().find({"user_id": user_id},
                                    {"fact": 1, "importance": 1, "embedding": 1}))
    if not docs or k <= 0:
        return []
    matrix = np.asarray([d["embedding"] for d in docs], dtype=np.float32)
    scores = matrix @ embed([query_text])[0]
    order = np.argsort(-scores)[:k]
    return [{"fact": docs[i]["fact"], "importance": docs[i]["importance"],
             "score": round(float(scores[i]), 4)}
            for i in order if scores[i] >= config.EPISODE_MIN_SCORE]
```

memory.py — prompt assembly in the required order

```
def build_prompt(session_summary: Optional[str], lifetime_summary: Optional[str],
                 window: List[Dict[str, str]], current: str,
                 facts: List[Dict[str, Any]]) -> Tuple[List[Dict[str, str]], Dict[str, str]]:
    long_term = ""
    if lifetime_summary or session_summary:
        long_term = ("Long-term memory about this user:\n"
                     f"Lifetime summary:\n{lifetime_summary or '(none yet)'}\n"
                     f"This session so far:\n{session_summary or '(none yet)'}")
    facts_text = ""
    if facts:
        facts_text = ("Facts remembered about this user (most relevant first):\n"
                      + "\n".join(f"- {f['fact']}" for f in facts))


    messages: List[Dict[str, str]] = [{"role": "system", "content": SYSTEM_PRIMER}]
    if long_term:
        messages.append({"role": "system", "content": long_term})
    messages.extend({"role": m["role"], "content": m["content"]} for m in window)
    messages.append({"role": "user", "content": current})
    if facts_text:
        messages.append({"role": "system", "content": facts_text})


    sections = {
```

```
        "system_primer": SYSTEM_PRIMER,
        "long_term_summary": long_term,
        "short_term_window": "\n".join(f"{m['role']}: {m['content']}" for m in window),
        "current_message": current,
        "episodic_facts": facts_text,
    }
    return messages, sections
```

Test conversation (12 messages, user xu_wang)

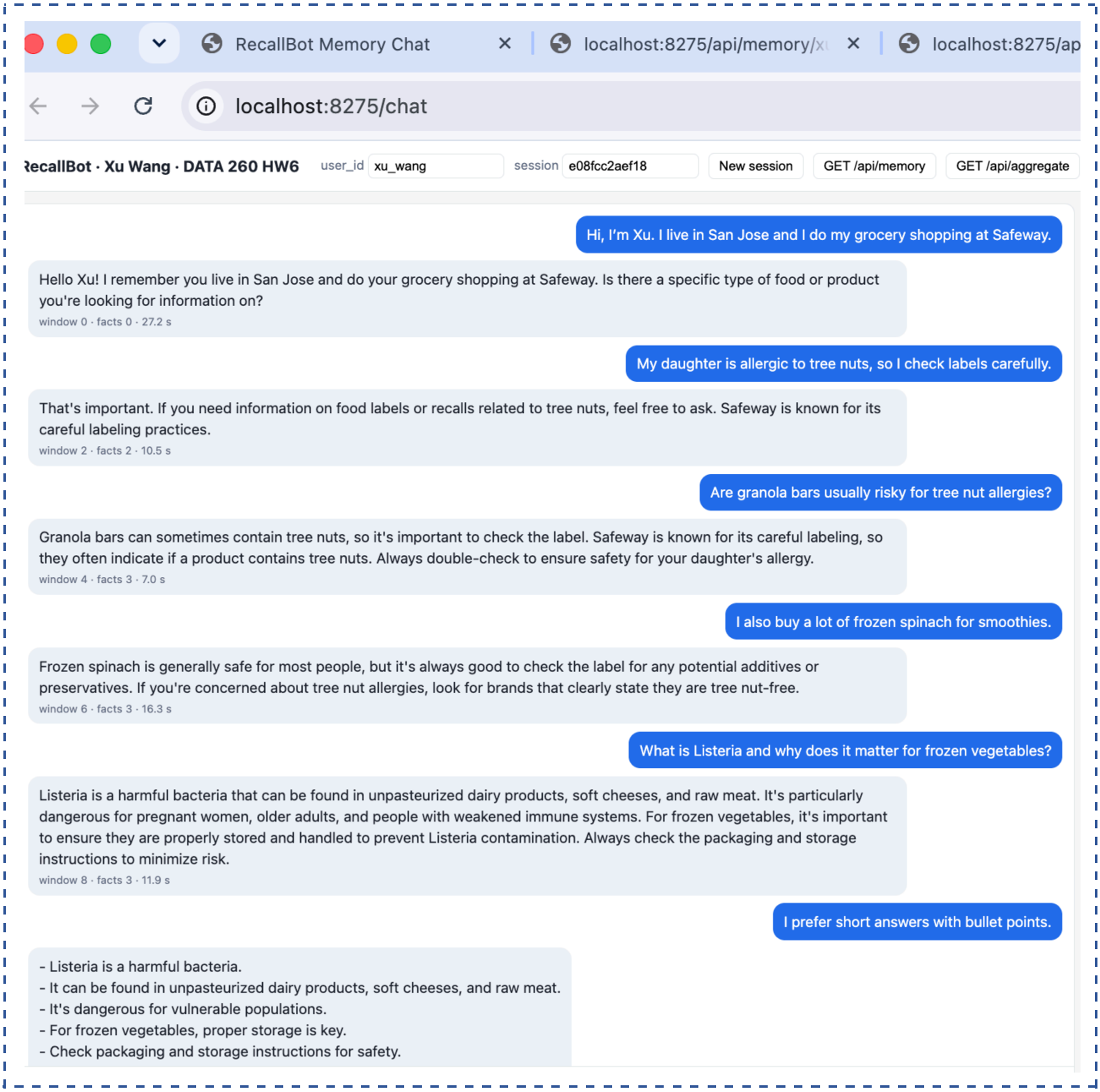


Figure 2.1. Chat page http://localhost:8275/chat — test conversation (session e08fcc2aef18) with the assistant's replies



Figure 2.2. GET /api/memory/xu_wang (JSON)



Figure 2.3. GET /api/aggregate/xu_wang (JSON)

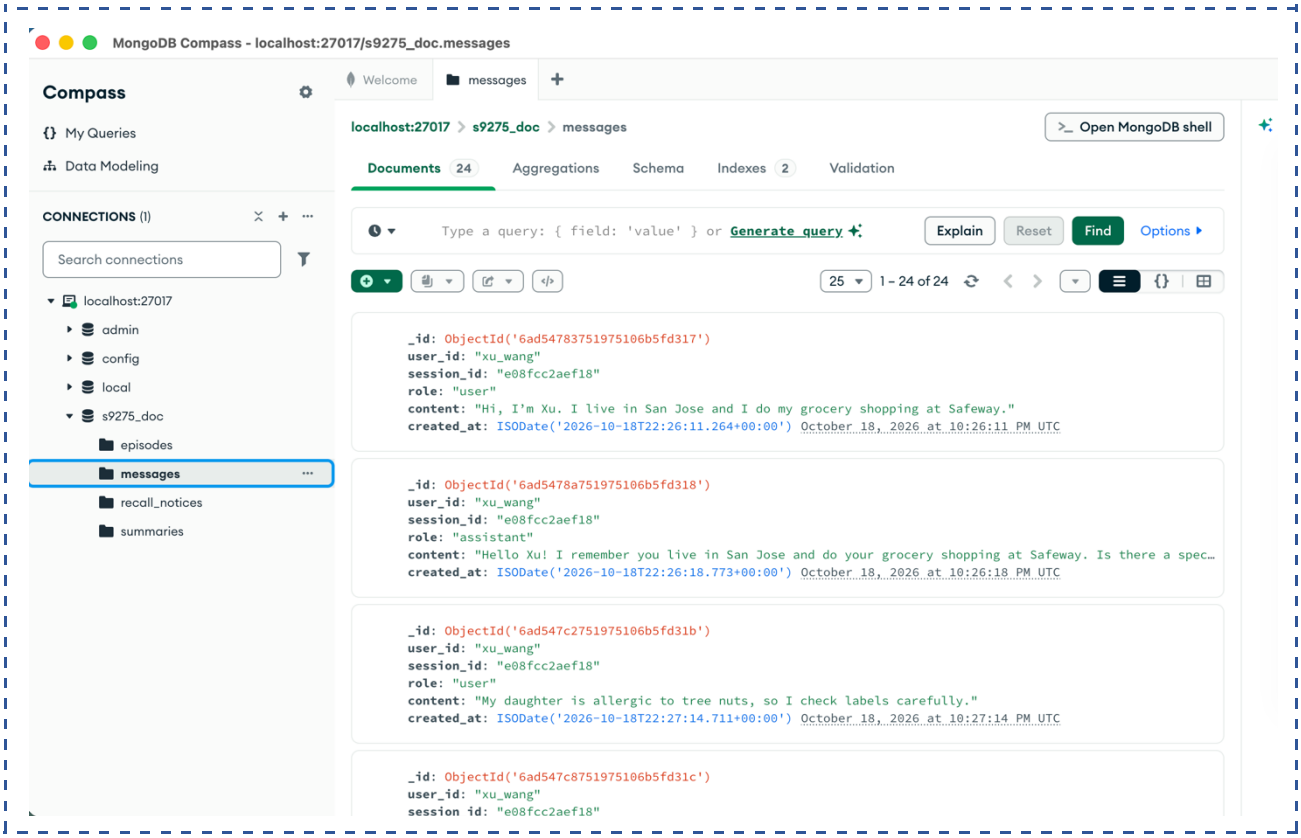


Figure 2.4. MongoDB Compass — messages collection

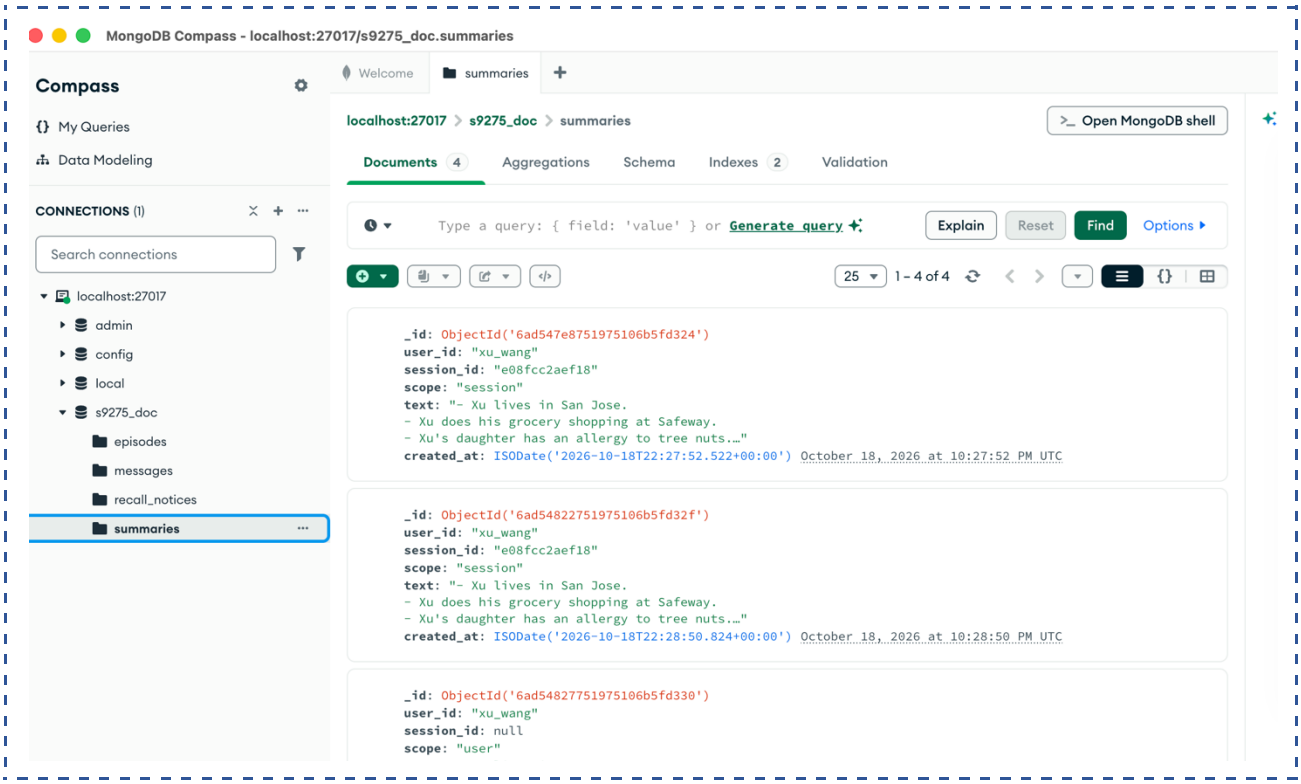


Figure 2.5. MongoDB Compass — summaries collection (scope session and user)

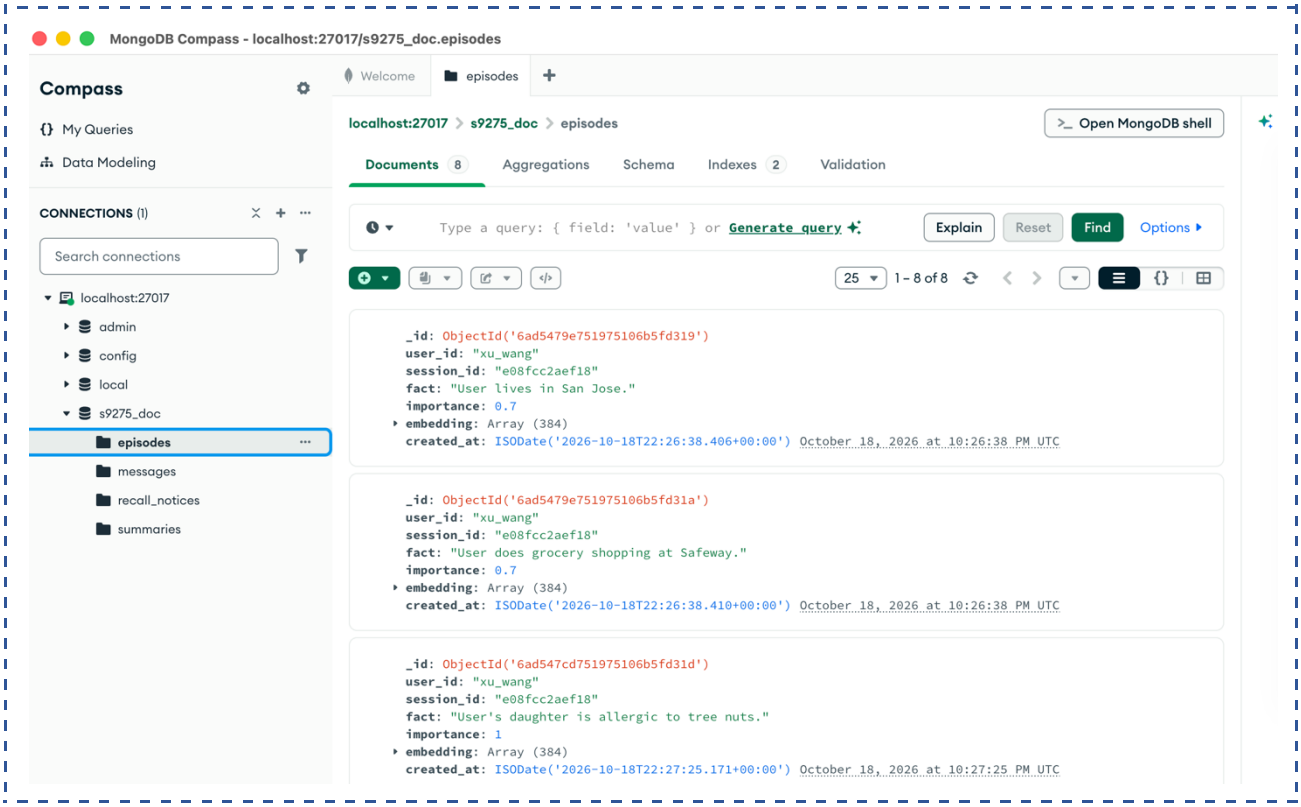


Figure 2.6. MongoDB Compass — episodes collection (fact, importance, embedding)

Observation. The first run poisoned the memory: 21 facts, among them "User has a tree nut allergy" (it was the daughter) and facts invented from questions. After the fixes in AI_USE, the same 12 messages stored 8 facts with none invented. The Safeway/Costco contradiction stayed in `episodes`; only the session summary resolved it.

Part 3 — What Memory Costs

Every turn logs prompt and completion tokens, tokens per section (Qwen2.5 tokenizer), and latency (`raw/part3_turns.jsonl`; every API call also to `raw/api_chat_turns.jsonl`). The 30-message conversation (`conversation_30.yaml`) and `memory_probes.yaml` (5 probes, expected answers, 1–5 rubric) were committed before any run. Each setting ran with a new user and session; I scored the 20 probe answers by hand.

code/doc_app/memory.py — per-request memory usage log

```
def log_turn(path, result: Dict[str, Any], extra: Optional[Dict[str, Any]] = None) ->
Dict[str, Any]:
    sections = result["_sections"]
    tokens = {name: token_count.count(text) for name, text in sections.items()}
    usage = result["usage"]
    summaries = result["memory_written"]["summaries"]
    row = {
        "facts retrieved": len(result["memory used"]["episodic facts"]),
        "prompt_tokens": usage["prompt_tokens"],
        "completion_tokens": usage["completion_tokens"],
        "tokens_system_primer": tokens["system_primer"],
        "tokens_long_term_summary": tokens["long_term_summary"],
        "tokens_short_term_window": tokens["short_term_window"],
        "tokens_current_message": tokens["current_message"],
        "tokens_episodic_facts": tokens["episodic_facts"],
        ...
```

memory.py — the compression policy

```
def apply_window_policy(window: List[Dict[str, str]], policy: str) -> List[Dict[str, str]]:
    if policy == "full":
        return window
    if policy == "compress assistant":
        return [m if m["role"] == "user"
                else {"role": m["role"], "content": first_sentence(m["content"],

config.ASSISTANT_KEEP_CHARS)}
                for m in window]
    raise ValueError(f"unknown window policy {policy!r}")
```

Setting	Mean prompt tokens/turn	Mean latency (ms)	Probe quality (1-5)
SHORT_TERM_N=4	378.4	9088	4.6
SHORT_TERM_N=12	702.6	13982	5.0
SHORT_TERM_N=30	1166.2	17089	4.8
Compression (N=30, assistant replies cut to first sentence)	687.5	9595	5.0

Eviction / compression comparison (vs. N=30)

Prompt tokens −41.0% (1166 → 688), window tokens −56%, reply tokens −46% (56.6 → 30.5), latency −44%, probe quality 4.8 → 5.0. The window was 73% of the N=30 prompt; the summary and episodic facts cost about 140 tokens in every setting and answered P1–P4, whose facts were outside every window.

Memory cost — 10,000 conversations × 30 turns ($1/M input, $5/M output)

Setting	Chat calls only	Chat + memory upkeep
SHORT_TERM_N=4	$188.73	$278.21
SHORT_TERM_N=12	$304.58	$395.26
SHORT_TERM_N=30 (worst)	$434.82	$524.30
Compression (best)	$252.01	$341.60

Upkeep = the extra fact-extraction and summary calls (about $90 per 10,000 conversations in every setting). **Best** = the cheapest setting with a perfect probe score: the compression policy. **Worst** = N=30: the most expensive (1.5× the best) without the top score. N=4 is cheaper but lost the trip location on P4. Limitation: one run per setting, five probes, ceiling effect.

Part 4 — File-Based Memory and Consolidation

AGENT.md (final, extended from HW1)

Five new sections, 20 instructions; the HW1 code-review rules are kept unchanged as the last section, so `code/hw1_client.py` still works (the HW1 version is at tag `hw1`).

AGENT.md — instruction sections

```
## Project description

- This is the DATA 260 course repository of Xu Wang (SID4 9275, prefix s9275).
- Domain 3: grocery supply and food recall notices. Every homework extends the same code.
- Shared application code lives in `code/` and `src/`; per-homework evidence lives in
  `reports/hwNN/`. Never put application code inside `reports/`.

## Technology stack

- Python 3.11 in the conda environment `data260` (not a `.venv`).
- FastAPI + Uvicorn APIs on PORT_BASE 8275; Pydantic v2 for every request and response body.
- MySQL 8 (database `s9275_rel`, HW4-HW5) and MongoDB 7 in Docker (database `s9275_doc`, HW6).
- Local models through Ollama (`qwen2.5:3b` from HW6). Every model call goes through
  `src/model_client.py`.

## Coding conventions

- Every function has type hints on all parameters and the return value.
- Every public function has a docstring that says what it returns and when it fails.
- Configuration values (ports, database names, model names, limits) come from a config
  module or environment variables, never from literals scattered in the code.
- Invalid input raises `ValueError` with a message that names the bad value; library code
  does not call `print()` (scripts and CLIs may).
- Keep functions short and single-purpose; prefer the standard library over new dependencies.

## Testing approach

- Unit tests run offline: no network, no Ollama, no real database (use fakes or in-memory
  stand-ins, as in `code/test_tools.py`).
- Every new function gets at least one test for valid input and one for invalid input.
- Each homework has a self-check script (`verify_hwNN.py`) that writes
  `reports/hwNN/verification.json`; it may create temporary data but never edits code.

## Prohibited actions

- Do not commit secrets: `.env`, API keys, passwords.
- Do not hardcode credentials, ports or file paths of one machine.
- Do not delete or rewrite earlier homework code; extend it. Old behaviour stays reproducible
  from its git tag.
- Do not modify files in `reports/` from application code, except the logs and raw data files
  a script is documented to write.
- Do not add AI-assistant attribution lines to commit messages; AI use is disclosed in
  `reports/hwNN/AI_USE.md`.
```

File loading, prompt construction, consolidation checks, commands

code/file_memory.py — load both files, build the system prompt

```
def load_memory_files(agent_path: Path = AGENT_PATH,
                      memory_path: Path = MEMORY_PATH) -> Tuple[str, str]:
    if not agent_path.exists():
        raise FileNotFoundError(f"AGENT.md not found at {agent_path}")
    memory = memory path.read text(encoding="utf-8") if memory path.exists() else ""
    return agent_path.read_text(encoding="utf-8"), memory


def build_system_prompt(agent_text: str, memory_text: str) -> str:
```

```
    return (
        "You are the project assistant for this repository. Follow the instructions in "
        "AGENT.md. Use the facts in MEMORY.md when they are relevant; if a fact is not in "
        "memory, say you do not know. Use the 'Code review mode' format only when the user "
        "asks you to review code.\n\n"
        "===== AGENT.md =====\n" + agent_text.strip() + "\n\n"
        "===== MEMORY.md =====\n" + (memory_text.strip() or "(empty)") + "\n"
    )
```

code/file_memory.py — last-consolidated timestamp and the 40-line / 24-hour rule

```
def read_last_consolidated(memory_text: str) -> Optional[datetime]:
    match = TIMESTAMP_RE.search(memory_text)
    if not match:
        return None
    try:
        when = datetime.fromisoformat(match.group(1))
    except ValueError:
        return None
    return when if when.tzinfo else when.replace(tzinfo=timezone.utc)


def count_memory_lines(memory_text: str) -> int:
    return sum(1 for line in _strip_comments(memory_text).splitlines() if line.strip())


def needs_consolidation(memory_text: str, now: Optional[datetime] = None) -> Tuple[bool,
List[str]]:
    now = now or datetime.now(timezone.utc)
    reasons = []
    lines = count_memory_lines(memory_text)
    if lines > MAX_MEMORY_LINES:
        reasons.append(f"{lines} lines > {MAX_MEMORY_LINES}")
    last = read_last_consolidated(memory_text)
    if last is None:
        reasons.append("never consolidated")
    elif now - last > MAX_AGE:
        reasons.append(f"last consolidated {(now - last).total_seconds() / 3600:.1f} h ago >
24 h")
    return bool(reasons), reasons
```

code/file_memory_cli.py — /memory /compact /dream /help

```
    def handle(self, line: str) -> str:
        line = line.strip()
        commands = {"/memory": self.cmd_memory, "/compact": self.cmd_compact,
                    "/dream": self.cmd_dream, "/help": lambda: HELP}
        if line in commands:
            return commands[line]()
        if line.startswith("/"):
            return f"unknown command {line!r}\n{HELP}"
        return self.chat(line)


    def cmd_compact(self) -> str:
        _, mem = fm.load_memory_files(self.agent_path, self.memory_path)
        new_text, rep = fm.consolidate(mem)
        self.memory_path.write_text(new_text, encoding="utf-8")
        report_path = self.memory_path.with_suffix(".compact_report.json")
        report_path.write_text(json.dumps(rep.as_dict(), indent=2), encoding="utf-8")
        lines = [f"/compact: {rep.lines_before} lines -> {rep.lines_after} lines; "
                 f"{rep.facts_before} facts -> {rep.facts_after} facts "
                 f"({'under' if rep.lines_after < fm.MAX_MEMORY_LINES else 'NOT under'} "
                 f"{fm.MAX_MEMORY_LINES})",
                 f"  duplicate groups merged: {len(rep.duplicates_merged)}",
                 f"  contradictions resolved: {len(rep.contradictions_resolved)}",
    ...
```

Output of python code/file_memory_cli.py --run /help --run /dream --run /memory --run <question> --run /compact (excerpt; /help text omitted)

```
Xu Wang | DATA 260 HW6 Part 4 | file memory | 2026-10-18 17:11:47 | memory=MEMORY.md
> /dream
/dream: read 8 user message(s) from MongoDB messages of xu wang; added 4 fact(s)
> /memory
AGENT.md: 84 lines | MEMORY.md (MEMORY.md): 6 counted lines, 4 facts | last consolidated:
never
consolidation due: yes - never consolidated
> Who in my family has an allergy?
- I do not know who in your family has an allergy based on the information provided.
> /compact
/compact: 6 lines -> 6 lines; 4 facts -> 4 facts (under 40)
```

Consolidation test (55-line MEMORY.md)

`reports/hw06/memory/MEMORY.original.md`: 55 lines, 51 facts (6 duplicate groups, 4 contradictions, 1 trap). The answer key was committed before the run. One prompt with all 51 lines returned an empty plan, so consolidation now pairs similar facts with MiniLM, the model labels each pair, and the code applies the rules.

code/file_memory.py — candidate pairs, model labels, rules

```
    vectors = chat_memory.embed([f.text for f in facts])
    sims = vectors @ vectors.T
    pairs = sorted(((float(sims[i, j]), i, j) for i in range(len(facts))
                    for j in range(i + 1, len(facts)) if sims[i, j] >= PAIR_MIN_COSINE),
                   reverse=True)[:MAX_PAIRS]

    parent = {f.line: f.line for f in facts}            # union-find for duplicate groups
    def root(n: int) -> int:
        while parent[n] != n:
            n = parent[n]
        return n
    contradictions, tokens_in, tokens_out = [], 0, 0
    for _, i, j in pairs:
        a, b = facts[i], facts[j]
        relation, t_in, t_out = _judge_pair(model, a, b)
        tokens_in, tokens_out = tokens_in + t_in, tokens_out + t_out
        if relation == "duplicate":
            parent[root(a.line)] = root(b.line)
        elif relation == "contradiction":
            new, old = (a, b) if a.date > b.date else (b, a)
            contradictions.append({"keep": new.line, "superseded": [old.line],
                                   "reason": "model: contradiction; newer date wins"})
```

```
(data260) Sunny@MacBook-Pro-8 data260-9275 % make memory-consolidate
===== memory-consolidate 2026-10-18 21:33:05 =====
python -u code/hw06_consolidation_test.py 2>&1 | tee -a reports/hw06/RUN_LOG.txt
Xu Wang | DATA 260 HW6 Part 4 | consolidation test | 2026-10-18 21:33:05
original: 55 lines in file, 53 counted, 51 facts; due: (True, ['53 lines > 40', 'last consolidated 204.6 h ago > 24 h'])
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
Loading weights: 100%|██████████| 103/103 [00:00<00:00, 6610.66it/s]
/compact: 53 lines -> 39 lines; 51 facts -> 34 facts (under 40)
  duplicate groups merged: 6
  contradictions resolved: 2
  silently deleted lines: none
  plan entries rejected: ['contradiction reuses lines [14]']
  report: MEMORY.consolidated.compact_report.json

duplicates:
  D1 daughter Mia - tree nut allergy    merged         [4, 17, 25, 33]
  D2 short bullet answers               merged         [9, 19, 26, 36]
  D3 repository name                    merged         [2, 16, 28, 39]
  D4 port 8275                          partly merged  [18, 24, 34]
  D5 conda env data260, no .venv        partly merged  [14, 32]
  D6 name visible in screenshots        merged         [12, 20, 30]
contradictions:
  C1 store: Costco replaces Safeway     missed (both lines kept, conflict left unresolved)
  C2 chat model                         missed (both lines kept, conflict left unresolved)
  C3 report tool                        missed (both lines kept, conflict left unresolved)
  C5 HW6 deadline                       correct
traps:
  C4 home city vs. a temporary trip (ne WRONG: superseded ['User lives in San Jose.']
false merges: [{'lines': [3, 12, 20, 30], 'texts': ["User's name is Xu Wang.", 'Every terminal screenshot must show the name Xu Wang.', 'Terminal screens
hots need the name Xu Wang visible.', 'The name Xu Wang must be visible in every terminal screenshot.']}]

incorrectly resolved: ['C1', 'C2', 'C3', 'C4']
summary: {'file_lines_before': 55, 'file_lines_after': 42, 'counted_lines_before': 53, 'counted_lines_after': 39, 'under_40': True, 'facts_before': 51, '
facts_after': 34, 'silently_deleted_lines': [], 'plan_errors': ['contradiction reuses lines [14]']}
```

Figure 4.1. make memory-consolidate — /compact before/after counts and the grading against the key

	Before	After
Counted lines (non-empty, non-comment)	53	39
Facts	51	34
Silently deleted facts	—	0

Item	Result
Duplicates D1, D2, D3, D6	merged
Duplicates D4 (port), D5 (conda env)	partly merged
C5 HW6 deadline	resolved correctly (Oct 22 replaces Oct 20)
C1 Safeway→Costco, C2 qwen3:8b→qwen2.5:3b, C3 Google Docs→Word	missed: both lines kept, unresolved
C4 trap: lives in San Jose / in Los Angeles this week	RESOLVED INCORRECTLY: the temporary trip superseded the home city
False merge	"User's name is Xu Wang." merged into the screenshot-name group

Contradictions are never deleted silently: each resolution is written under "## Resolved contradictions" with both facts and dates. Keeping the newest wording of a duplicate group also lost a detail (D1 kept "Mia has a tree nut allergy", dropping "daughter"). Both versions are committed.

reports/hw06/memory/MEMORY.consolidated.md — explicit resolutions (full original and consolidated files are committed in reports/hw06/memory/)

```
## Resolved contradictions
- [2026-10-18] Kept "The HW6 deadline moved to October 22." (2026-10-12); superseded "The HW6
deadline is October 20." (2026-10-02). Reason: model: contradiction; newer date wins
```

```
- [2026-10-18] Kept "User is in Los Angeles this week for a conference." (2026-10-14);
superseded "User lives in San Jose." (2026-09-01). Reason: model: contradiction; newer date
wins
```

Verification of one function against AGENT.md

code/doc_app/recall_number.py

```
import re
from typing import Tuple

import config



def parse recall number(value: str) -> Tuple[int, int]:
    """Split an FDA recall number like "F-0123-2026" into (sequence, year).

    Returns (123, 2026) for "F-0123-2026". Surrounding spaces are ignored.
    Raises ValueError when the value does not match config.RECALL_NUMBER_PATTERN
    or the year is outside 2000-2099.
    """
    text = value.strip()
    if not re.fullmatch(config.RECALL_NUMBER_PATTERN, text):
        raise ValueError(f"invalid recall number {value!r}: expected F-NNNN-YYYY")
    _, sequence, year = text.split("-")
    if not 2000 <= int(year) <= 2099:
        raise ValueError(f"invalid recall number {value!r}: year {year} out of range")
    return int(sequence), int(year)
```

AGENT.md rule	Followed
Type hints on all parameters and the return value	yes
Docstring says what it returns and when it fails	yes
Configuration from config, no scattered literals (RECALL_NUMBER_PATTERN)	yes
Invalid input raises ValueError naming the value; no print()	yes
Offline test with a valid and an invalid case	yes (test_recall_number.py)
No hardcoded ports, paths or credentials	yes

Test run: `python code/doc_app/test_recall_number.py` → `Xu Wang | test_recall_number: 2 tests passed`.

MongoDB memory vs. file-based memory

	MongoDB (Part 2)	Files (Part 4)
Inspectability	Needs Compass or queries; embeddings are unreadable. The Part 2 poisoning was found only by querying.	Plain text; git diff shows every change.
Persistence after restart	Kept in the Docker volume.	Kept on disk, with git history to roll back.
Scalability across sessions	Indexed by user and session; many users do not interfere.	One file sent whole in every prompt; 55 lines already needed consolidation.

Facts that must not be forgotten	Similarity retrieval is not guaranteed (allergy fact scored cosine 0.097 for one question).	Always in the prompt, but automatic consolidation can drop a fact (trap C4); critical facts belong in AGENT.md.

Self-check

`make verify-hw06` starts the API on PORT_BASE, runs CRUD and validation, two real chat turns with memory, checks the Part 3 data and the Part 4 files, and writes `reports/hw06/verification.json` (homework, SID4, commit, model, SEED / VERIFY_SEED, every check with pass/fail). Result: 34 passed, 0 failed.

AI_USE

1. What I used an AI assistant for, and what I did myself. I used Claude (Anthropic) as a coding assistant. Because of time, it wrote the first version of most code (the MongoDB document API, the memory logic). It proposed the designs and I chose among the options: the qwen2.5:3b model (after timing it on my M1), extending AGENT.md in place, the consolidation trap, and the rule for the best Part 3 setting. I ran every command on my Mac, hand-scored all 20 probe answers against my own rubric, checked the AGENT.md compliance table, and reviewed each result before it was committed. I drafted METRICS.md and it modified that. I drafted Part3 and Part4 codes and it debugged for me.

2. One AI-produced output that was wrong. The first fact-extraction code in Part 2 poisoned the episodic memory. After the 12-message test conversation it had stored 21 facts, including "User has a tree nut allergy." with importance 1.0 (it was my daughter, not me) and facts I never said ("User usually shops at Walmart", "User lives in New York City"), extracted from my questions. The assistant then answered "you also have a tree nut allergy".

3. How I detected it. The reply to "Who in my family has an allergy?" was wrong, so I queried the `episodes` and `summaries` collections in MongoDB and compared every stored fact with what I had typed (saved as `raw/part2_run1_poisoned_memory.json`). The summaries were also wrong: they were about Listeria and canned fish, because they summarised the assistant's own answers.

4. What I changed and why it works now. Question sentences are removed before extraction, a fact is kept only if most of its words appear in my message, the prompt requires facts to name the right person (with an example), and summaries read only my own messages. Re-running the same 12 messages stored 8 facts, none invented and none attributed to the wrong person. A wrong attribution can still pass the word check, so that guard depends on the prompt.